Bản báo cáo lab 3

I. RTOS (Real-time Operating System - Hệ điều hành thời gian thực)

a. Khái niệm

– Là một hệ điều hành nhằm phục vụ các ứng dụng thời gian thực, với việc xử lý dữ liệu đầu vào mà không có sự chậm trễ của bộ đệm. Các yêu cầu về thời gian xử lý được tính bằng phần mười của giây hoặc bằng thời gian ngắn hơn.

– Quá trình xử lý trong RTOS phải diễn ra trong các ràng buộc, giới hạn thời gian được chỉ định, nếu không sẽ dẫn đến lỗi hệ thống. RTOS hoạt động dựa trên hai cơ chế là hướng sự kiện hoặc chia sẻ thời gian (time-sharing):

+ Cơ chế hướng sự kiện sẽ giải quyết và điều phối các tác vụ (task) thông quamức độ ưu tiên của chúng.

+ Cơ chế chia sẻ thời gian sẽ chuyển đối các tác vụ dựa trên phản ứng ngắt củaxung nhịp.

– Phần lớn các hệ điều hành thời gian thực đều sử dụng giải thuật Pre-emptiveScheduling. RTOS hỗ trợ xây dựng các ứng dụng đáp ứng thời gian thực để cung cấp các kết quả tính toán chính xác về mặt logic

b. Ưu điểm

– Đáp ứng đúng hạn: task ưu tiên cao được chạy ngay khi sẵn sàng, nên các việc khẩn cấp như phát hiện té ngã không bị việc khác làm chậm.

– Chia chương trình theo chức năng: mỗi việc là một task riêng, code gọn, dễ hiểu, dễ sửa và dễ làm việc nhóm.

– Tận dụng nhiều nhân: như ESP32 có 2 nhân, có thể gắn task vào từng Core.

– Không bị chặn bởi delay(): task chờ thì nhường CPU cho task khác, nên WiFi chậm không làm treo việc đọc cảm biến.

– Tiết kiệm CPU: task chờ sự kiện ở trạng thái blocked, không tốn CPU như vòng lặp kiểm tra liên tục.

– Có sẵn cơ chế đồng bộ và giao tiếp: queue, semaphore, mutex, timer.

– Dễ mở rộng: thêm tính năng mới bằng cách thêm task, ít ảnh hưởng phần cũ.

c. Ưu/Nhược điểm của FreeRTOS

– Ưu điểm:

+ Miễn phí, mã nguồn mở, dùng được cho cả sản phẩm thương mại.

+ Nhỏ gọn: nhân chạy được trên vi điều khiển rất ít RAM.

+ Phổ biến: hỗ trợ hàng chục kiến trúc (ARM, ESP32, AVR...), nhiều tài liệu và ví dụ.
+ Tích hợp sẵn trên ESP32, hỗ trợ chạy 2 nhân.
+ Đủ tính năng cơ bản: task, queue, semaphore, mutex, software timer, event group.
– Dễ kết nối với dịch vụ đám mây AWS.
– Nhược điểm:
+ Là nhân lập lịch: không có hệ thống file, mạng, giao diện... như Linux, phải tự ghép thư viện.
+ Không có bảo vệ bộ nhớ, một task lỗi có thể làm hỏng cả hệ thống.
+ Gỡ lỗi khó: lỗi tranh chấp, tràn stack, bế tắc khó tái hiện.
+ Phải tự tính stack và ưu tiên cho từng task; chọn sai gây treo hoặc tụt hiệu năng.
+ Lập lịch đơn giản, không có các thuật toán nâng cao như EDF.
+ Cần kinh nghiệm để thiết kế đúng so với cách viết loop() thông thường.

II. Cơ chế giao tiếp giữa các task

a. Queue (Hàng đợi)

– Là cơ chế giao tiếp giữa các tác vụ trong FreeRTOS. Nó cho phép một task gửi dữ liệu cho task khác, hoặc giữa một ngắt và một task.freertos
– Hoạt động theo kiểu FIFO (First In First Out – vào trước ra trước): dữ liệu gửi vào cuối hàng đợi, task nhận lấy dữ liệu từ đầu hàng đợi.freertos
– Dữ liệu được gửi qua queue theo kiểu copy, nghĩa là FreeRTOS tự sao chép nội dung vào vùng nhớ của queue, không phải chỉ lưu con trỏ.freertos
– Có thể cấu hình queue chứa nhiều phần tử, mỗi phần tử có kích thước cố định (ví dụ: int, struct, con trỏ...).
– Ưu điểm
+ An toàn khi dùng giữa nhiều task (thread-safe): FreeRTOS tự quản lý việc truy cập, không cần tự viết khóa.
+ Truyền dữ liệu thực sự, không chỉ báo hiệu: có thể gửi số, struct, con trỏ...
+ Linh hoạt về kích thước: có thể gửi dữ liệu nhỏ trực tiếp, hoặc gửi con trỏ cho dữ liệu lớn.
+ Hỗ trợ nhiều loại tin nhắn trong cùng một queue bằng cách dùng struct chứa trường "loại tin nhắn".
+ Tự động quản lý bộ nhớ cho vùng lưu trữ queue.

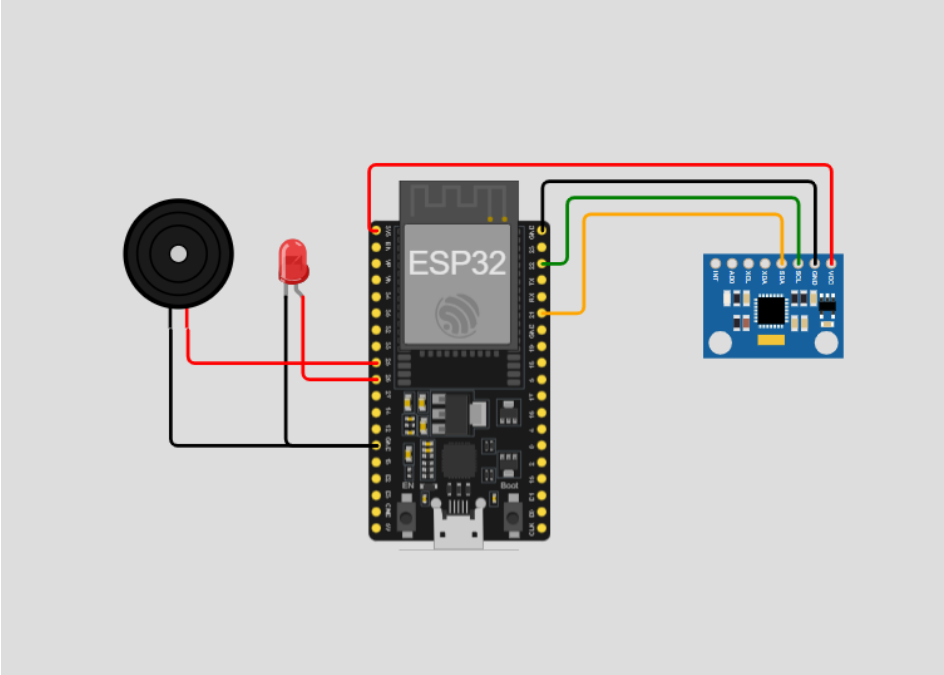
+ Có API riêng cho ISR (xQueueSendFromISR, xQueueReceiveFromISR) để dùng an toàn trong ngắt.
– Ứng dụng:
+ Gửi dữ liệu cảm biến từ ISR sang task xử lý.
+ Truyền thông điệp giữa các task
+ Xử lý nhiều loại sự kiện khác nhau qua một queue duy nhất (dùng struct có trường "message type").
+ Hàng đợi lệnh điều khiển

b. Semaphore (Cờ hiệu)

– Là cơ chế đồng bộ hóa hoặc loại trừ lẫn nhau giữa các task, hoặc giữa task và ngắt. Trong FreeRTOS, semaphore được xây dựng dựa trên queue.
– Có 2 loại chính:
+ Binary semaphore (cờ hiệu nhị phân): chỉ có 2 trạng thái – "có" (1) hoặc "không" (0).
+ Counting semaphore (cờ hiệu đếm): có thể đếm được nhiều lần, giống như queue có độ dài > 1.
– Ưu điểm:
+ Nhẹ và nhanh: phù hợp cho việc đồng bộ hóa đơn giản, đặc biệt giữa ISR và task.
+ Không cần truyền dữ liệu: chỉ cần báo hiệu "có việc cần làm" hoặc "tài nguyên đã sẵn sàng".
+ Tiết kiệm CPU: task có thể block chờ semaphore thay vì poll liên tục, giúp CPU làm việc khác.
+ Có API riêng cho ISR (xSemaphoreGiveFromISR) để dùng an toàn trong ngắt.
+ Counting semaphore cho phép đếm nhiều sự kiện cùng lúc (ví dụ: đếm số lần ngắt xảy ra).
– Ứng dụng:
+ Đồng bộ hóa giữa ISR và task: ISR "give" semaphore khi có sự kiện, task "take" để xử lý.
+ Báo hiệu tài nguyên sẵn sàng (ví dụ: dữ liệu UART đã nhận xong).
+ Đếm số lần ngắt xảy ra (dùng counting semaphore).
+ Đồng bộ hóa giữa 2 task

III. Thực hiện lab

- Sơ đồ mạch



Linh kiện	ESP32	Chân
MPU6050	VIN	3V3
MPU6050	GND	GND
MPU6050	SDA	GPIO21
MPU6050	SCL	GPIO22
Buzzer	(+)	GPIO25
Buzzer	(-)	GND
LED	(+)	GPIO26
LED	(-)	GND

- Bảng dữ liệu các task

Thành phần	Core	Việc làm
TaskTruyenData	0	Kết nối WiFi và MQTT. Khi thành công mới đọc dataQueue và gửi lên broker
TaskDocData	1	Đọc MPU6050, in Serial, đưa dữ liệu đầy đủ vào dataQueue và trạng thái cảnh báo vào alertQueue
TaskCanhBao	1	Đọc alertQueue, điều khiển còi và đèn

- Code

```
#include <WiFi.h>
#include <Wire.h>
#include <PubSubClient.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

// ====== CAU HINH ======
const char* WIFI_SSID = "Wokwi-GUEST";   // WiFi that: doi lai ten/mat khau
const char* WIFI_PASS = "";
const char* MQTT_HOST = "broker.hivemq.com";
const int   MQTT_PORT = 1883;

#define PREFIX "fall_demo_5a7c/"          // <-- DOI thanh ten rieng, trung voi index.html
const char* T_DATA = PREFIX "data";
const char* T_LED  = PREFIX "cmd/led";
const char* T_BUZ  = PREFIX "cmd/buzzer";
const char* T_RST  = PREFIX "cmd/reset";

#define PIN_LED    26
#define PIN_BUZZER 25

// ====== NGUONG THUAT TOAN TE NGA ======
```

```
const float FREEFALL_G        = 0.5;    // |a| < 0.5 g  -> roi tu do
const float IMPACT_G          = 2.0;    // |a| > 2.0 g  -> va cham manh
const float TILT_JUMP_DEG     = 50.0;   // goc doi > 50 do trong ~0.5 s -> nghieng dot ngot
const float TILT_LYING_DEG    = 60.0;   // goc > 60 do -> dang nam
const unsigned long FREEFALL_WINDOW_MS = 1000;  // sau roi tu do phai co va cham trong 1 s
const unsigned long CONFIRM_MS        = 1500;  // cho 1.5 s sau va cham roi xac nhan

const uint32_t SAMPLE_PERIOD_MS  = 100;   // doc cam bien 10 Hz
const uint32_t PUBLISH_PERIOD_MS = 500;   // in Serial + gui queue moi 0.5 s

// ====== KIEU DU LIEU ======
enum State : uint8_t { NORMAL, FREEFALL, IMPACT_WAIT, FALL };
const char* stateName[] = { "NORMAL", "FREEFALL", "IMPACT", "FALL" };

// Goi du lieu day du: Task doc data -> dataQueue -> Task truyen data
struct SensorData {
  float ax, ay, az;     // g
  float aTotal;         // g
  float tilt;           // do
  uint8_t state;        // State
};
```

```
// Ban tin canh bao: Task doc data / MQTT callback -> alertQueue -> Task canh bao
enum AlertType : uint8_t {
  ALERT_STATE,          // value = State (tu thuat toan)
  CMD_LED,              // value = 0/1 (manual)
  CMD_BUZZER            // value = 0/1 (manual)
};
struct AlertMsg {
  AlertType type;
  uint8_t   value;
};

// ====== DOI TUONG DUNG CHUNG ======
QueueHandle_t dataQueue  = NULL;   // "data Queue"
QueueHandle_t alertQueue = NULL;   // "data Alert"

Adafruit_MPU6050 mpu;
WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

volatile bool ledOn = false;          // chi TaskCanhBao ghi
volatile bool buzOn = false;          // chi TaskCanhBao ghi
volatile bool resetRequest = false;   // MQTT callback ghi, TaskDocData doc roi xoa
```

```
// ====== TASK CANH BAO (Core 1) ======
void setOutputs(bool led, bool buz) {
  ledOn = led;
  buzOn = buz;
  digitalWrite(PIN_LED, led);
  if (buz) tone(PIN_BUZZER, 2000);
  else     noTone(PIN_BUZZER);
}

void TaskCanhBao(void* pv) {
  AlertMsg msg;
  uint8_t lastState = NORMAL;

  for (;;) {
    if (xQueueReceive(alertQueue, &msg, portMAX_DELAY) != pdTRUE)
continue;

    switch (msg.type) {
      case ALERT_STATE:
        if (msg.value == FALL) {
          setOutputs(true, true);                // tu dong bat Den + Coi
        } else if (lastState == FALL) {
          setOutputs(false, false);              // het canh bao (Reset)
        }
```

```
        lastState = msg.value;
        break;

      case CMD_LED:                          // che do manual
        setOutputs(msg.value, buzOn);
        break;

      case CMD_BUZZER:
        setOutputs(ledOn, msg.value);
        break;
    }
  }
}

// ====== TASK DOC DATA (Core 1) ======
State state = NORMAL;
unsigned long tState = 0;
float tiltHist[5];
int   tiltIdx = 0;

// Tinh gia toc + goc nghieng tu MPU6050
void readSensor(SensorData& d, float& tiltDelta) {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);
```

```
  d.ax = a.acceleration.x / 9.80665;   // m/s^2 -> g
  d.ay = a.acceleration.y / 9.80665;
  d.az = a.acceleration.z / 9.80665;
  d.aTotal = sqrt(d.ax * d.ax + d.ay * d.ay + d.az * d.az);

  // Goc nghieng cua truc Z so voi phuong thang dung (0 do = dung thang)
  if (d.aTotal > 0.1) d.tilt = degrees(acos(constrain(d.az / d.aTotal, -1.0, 1.0)));

  // So sanh voi goc cach day ~0.5 s (5 mau x 100 ms)
  tiltDelta = fabs(d.tilt - tiltHist[tiltIdx]);
  tiltHist[tiltIdx] = d.tilt;
  tiltIdx = (tiltIdx + 1) % 5;
}

// May trang thai te nga; tra ve true neu trang thai vua doi
bool detectFall(const SensorData& d, float tiltDelta) {
  unsigned long now = millis();
  bool impact     = d.aTotal > IMPACT_G;
  bool suddenTilt = tiltDelta > TILT_JUMP_DEG;
  State old = state;

  switch (state) {
    case NORMAL:
      if (d.aTotal < FREEFALL_G)         { state = FREEFALL;    tState = now; }
```

```
    else if (impact || suddenTilt)      { state = IMPACT_WAIT; tState = now; }
    break;


  case FREEFALL:
    if (impact || suddenTilt)                  { state = IMPACT_WAIT; tState = now;
}
    else if (now - tState > FREEFALL_WINDOW_MS)  state = NORMAL;    //
roi ngan, khong va cham
    break;


  case IMPACT_WAIT:
    if (now - tState >= CONFIRM_MS) {
      // Xac nhan: dang nam (goc lon) va nam yen (|a| ~ 1 g)
      if (d.tilt > TILT_LYING_DEG && fabs(d.aTotal - 1.0) < 0.3) {
        state = FALL;
        Serial.println("!!! PHAT HIEN TE NGA !!!");
      } else {
        state = NORMAL;                        // bao dong gia
      }
    }
    break;


  case FALL:
    break;   // giu canh bao den khi nhan lenh Reset tu Web
 }
```

```
  return state != old;
}

void TaskDocData(void* pv) {
  SensorData d = {};
  d.aTotal = 1.0;
  float tiltDelta = 0;
  uint32_t lastPublish = 0;
  TickType_t lastWake = xTaskGetTickCount();

  readSensor(d, tiltDelta);
  for (int i = 0; i < 5; i++) tiltHist[i] = d.tilt;

  for (;;) {
    vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(SAMPLE_PERIOD_MS));

    readSensor(d, tiltDelta);

    // Lenh Reset tu Web
    bool changed = false;
    if (resetRequest) {
      resetRequest = false;
      state = NORMAL;
      changed = true;
```

```
    }
    changed |= detectFall(d, tiltDelta);
    d.state = state;

    // Trang thai doi -> bao cho Task canh bao
    if (changed) {
      AlertMsg am = { ALERT_STATE, (uint8_t)state };
      xQueueSend(alertQueue, &am, 0);
    }

    // Moi 0.5 s: in Serial + dua du lieu day du vao dataQueue
    uint32_t now = millis();
    if (now - lastPublish >= PUBLISH_PERIOD_MS || changed) {
      lastPublish = now;
      Serial.printf("ax=%.2f ay=%.2f az=%.2f |a|=%.2f g | tilt=%.1f do | %s |
LED=%d BUZ=%d\n",
                    d.ax, d.ay, d.az, d.aTotal, d.tilt, stateName[d.state], ledOn, buzOn);
      xQueueSend(dataQueue, &d, 0);   // day day thi bo qua (mang cham), khong
chan task
    }
  }
}

// ====== TASK TRUYEN DATA (Core 0) ======
void onMessage(char* topic, byte* payload, unsigned int len) {
```

```
  String msg;
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
  msg.trim();
  msg.toUpperCase();
  uint8_t on = (msg == "ON" || msg == "1") ? 1 : 0;
  String t(topic);

  AlertMsg am;
  if (t == T_LED)      { am = { CMD_LED, on };    xQueueSend(alertQueue,
&am, 0); }
  else if (t == T_BUZ) { am = { CMD_BUZZER, on };
xQueueSend(alertQueue, &am, 0); }
  else if (t == T_RST) { resetRequest = true; }
}

bool mqttConnect() {
  String id = "esp32-fall-" + String((uint32_t)ESP.getEfuseMac(), HEX);
  if (!mqtt.connect(id.c_str())) {
    Serial.printf("MQTT loi, rc=%d\n", mqtt.state());
    return false;
  }
  mqtt.subscribe(T_LED);
  mqtt.subscribe(T_BUZ);
  mqtt.subscribe(T_RST);
  Serial.println("MQTT da ket noi");
```

```
  return true;
}

void publishData(const SensorData& d) {
  char buf[220];
  snprintf(buf, sizeof(buf),
           "{\"ax\":%.2f,\"ay\":%.2f,\"az\":%.2f,\"a\":%.2f,\"tilt\":%.1f,"
           "\"state\":\"%s\",\"led\":%d,\"buzzer\":%d}",
           d.ax, d.ay, d.az, d.aTotal, d.tilt, stateName[d.state], (int)ledOn,
(int)buzOn);
  mqtt.publish(T_DATA, buf);
}

void TaskTruyenData(void* pv) {
  SensorData d;
  uint32_t tReconnect = 0;

  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setBufferSize(512);
  mqtt.setCallback(onMessage);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS, 6);          // kenh 6 cho Wokwi-GUEST
  Serial.println("Dang ket noi WiFi...");
```

```
  for (;;) {
    // 1) Chua co WiFi: cho, tuyet doi khong gui du lieu
    if (WiFi.status() != WL_CONNECTED) {
      vTaskDelay(pdMS_TO_TICKS(300));
      continue;
    }

    // 2) Co WiFi: dam bao ket noi MQTT
    if (!mqtt.connected()) {
      if (millis() - tReconnect >= 3000) {
        tReconnect = millis();
        mqttConnect();
      }
      vTaskDelay(pdMS_TO_TICKS(100));
      continue;
    }

    // 3) WiFi + MQTT thanh cong: doc dataQueue va gui len broker
    mqtt.loop();
    if (xQueueReceive(dataQueue, &d, pdMS_TO_TICKS(50)) == pdTRUE) {
      publishData(d);
    }
  }
}
```

```
// ====== SETUP / LOOP ======
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  setOutputs(false, false);

  Wire.begin(21, 22);
  if (!mpu.begin()) {
    Serial.println("Khong tim thay MPU6050! Kiem tra day SDA/SCL.");
    while (true) delay(1000);
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_16_G);   // mac dinh +-2g
se cat mat va cham
  mpu.setFilterBandwidth(MPU6050_BAND_44_HZ);

  // Tao hang doi
  dataQueue  = xQueueCreate(10, sizeof(SensorData));
  alertQueue = xQueueCreate(10, sizeof(AlertMsg));
  if (!dataQueue || !alertQueue) {
    Serial.println("Khong tao duoc Queue!");
    while (true) delay(1000);
  }
```

```
  // Tao task: (ham, ten, stack, tham so, uu tien, handle, core)
  xTaskCreatePinnedToCore(TaskTruyenData, "TruyenData", 8192, NULL, 1, NULL, 0);  // Core 0
  xTaskCreatePinnedToCore(TaskDocData,    "DocData",    4096, NULL, 2, NULL, 1);  // Core 1
  xTaskCreatePinnedToCore(TaskCanhBao,    "CanhBao",    2048, NULL, 3, NULL, 1);  // Core 1
}

void loop() {
  vTaskDelay(pdMS_TO_TICKS(1000));   // moi viec do cac task dam nhan
}
```

- Giải thích code

– Khai báo và cấu hình: Phần đầu code khai báo WiFi, địa chỉ broker HiveMQ, PREFIX để tạo các topic MQTT, chân LED/Buzzer và các ngưỡng té ngã (0,5 g, 2 g, 50°, 60°, 1,5 s). Hai kiểu dữ liệu SensorData và AlertMsg dùng để truyền tin qua hàng đợi.

– Hàm setup(): Khởi tạo Serial, chân LED/Buzzer, I2C và MPU6050 (thang đo ±16 g), rồi tạo hai hàng đợi dataQueue và alertQueue. Cuối cùng tạo 3 task bằng xTaskCreatePinnedToCore: Truyền data chạy Core 0, Đọc data và Cảnh báo chạy Core 1.

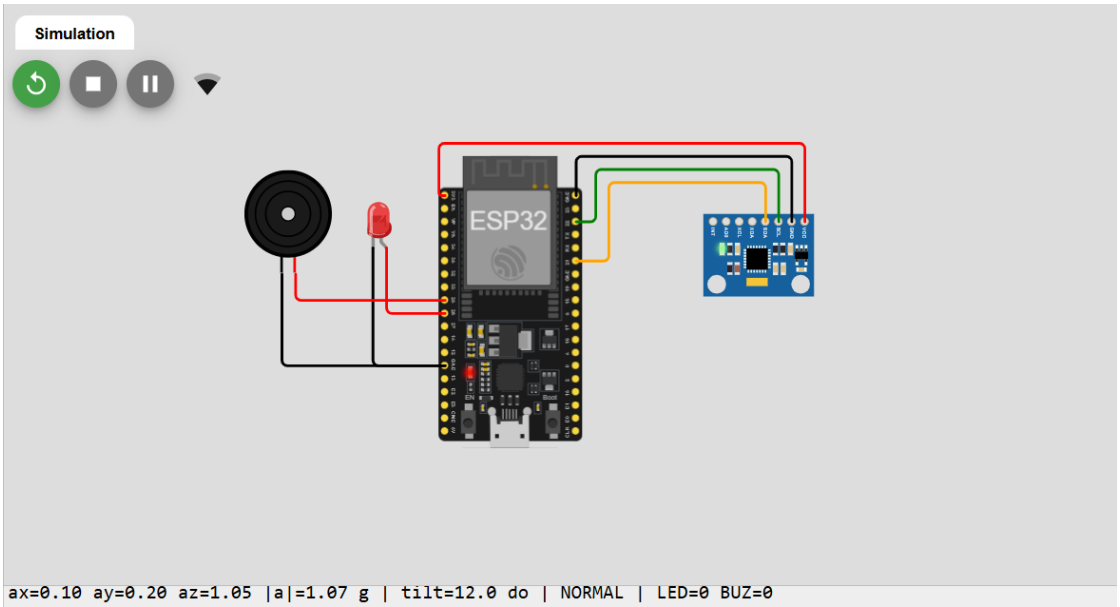
– TaskDocData (Core 1): Cứ 100 ms đọc MPU6050, đổi sang g, tính gia tốc tổng và góc nghiêng, rồi chạy máy trạng thái NORMAL → FREEFALL → IMPACT → FALL. Cứ 0,5 giây in ra Serial và đưa dữ liệu vào dataQueue, còn khi trạng thái đổi thì gửi vào alertQueue.

– TaskTruyenData (Core 0): Kết nối WiFi, sau đó kết nối MQTT broker và đăng ký các topic lệnh. Chỉ khi cả WiFi và MQTT thành công mới lấy dữ liệu từ dataQueue, đóng gói JSON và gửi lên topic data. Hàm onMessage nhận lệnh từ web và chuyển vào alertQueue, riêng lệnh Reset dùng cờ resetRequest.

– TaskCanhBao (Core 1): Nằm chờ trên alertQueue, khi có tin thì xử lý ngay: trạng thái FALL tự bật Đèn và Còi, lệnh CMD_LED/CMD_BUZZER từ nút web thì bật tắt thủ công. Đây là task duy nhất được ghi chân LED/Buzzer nên chế độ tự động và thủ công không xung đột.

IV. Kết quả

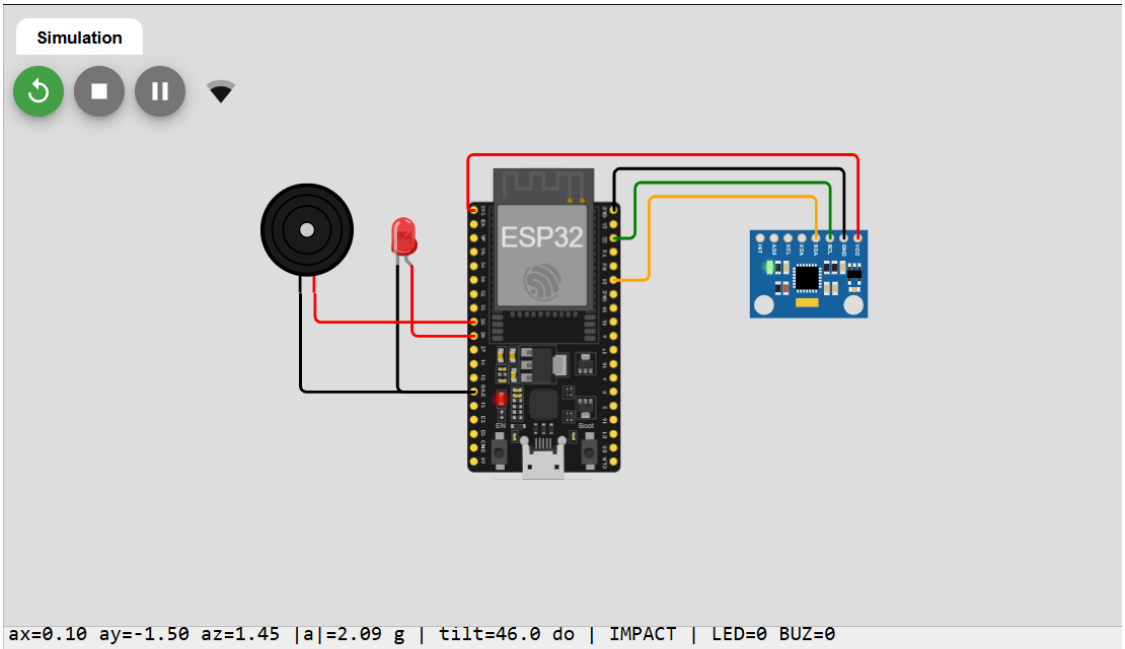
a. Trạng thái bình thường




```
ax=0.10 ay=0.20 az=1.05 |a|=1.07 g | tilt=12.0 do | NORMAL | LED=0 BUZ=0
ax=0.10 ay=0.20 az=1.05 |a|=1.07 g | tilt=12.0 do | NORMAL | LED=0 BUZ=0
ax=0.10 ay=0.20 az=1.05 |a|=1.07 g | tilt=12.0 do | NORMAL | LED=0 BUZ=0
ax=0.10 ay=0.20 az=1.05 |a|=1.07 g | tilt=12.0 do | NORMAL | LED=0 BUZ=0
ax=0.10 ay=0.20 az=1.05 |a|=1.07 g | tilt=12.0 do | NORMAL | LED=0 BUZ=0
ax=0.10 ay=0.20 az=1.05 |a|=1.07 g | tilt=12.0 do | NORMAL | LED=0 BUZ=0
ax=0.10 ay=0.20 az=1.05 |a|=1.07 g | tilt=12.0 do | NORMAL | LED=0 BUZ=0
```



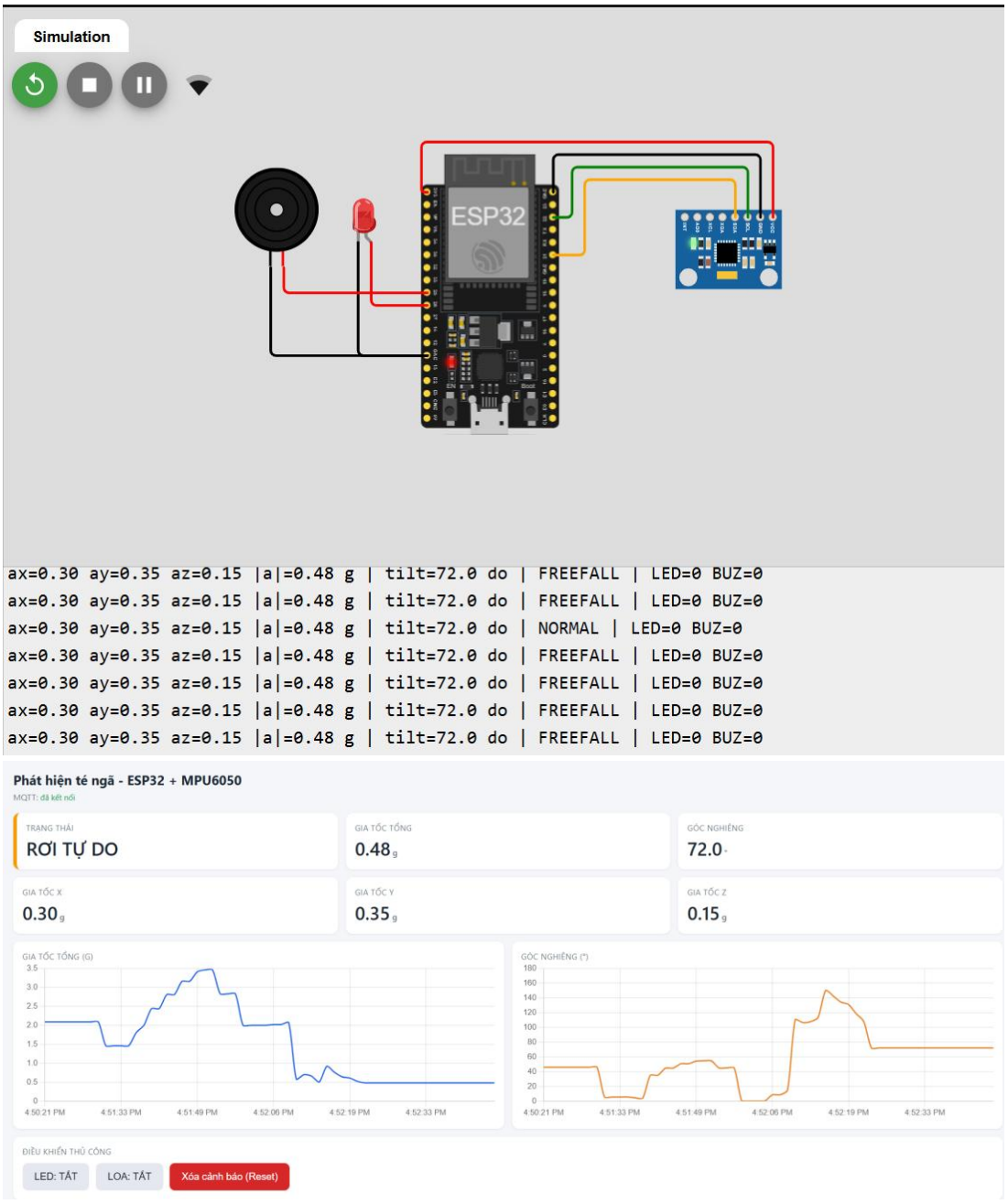
b. Trạng thái va chạm



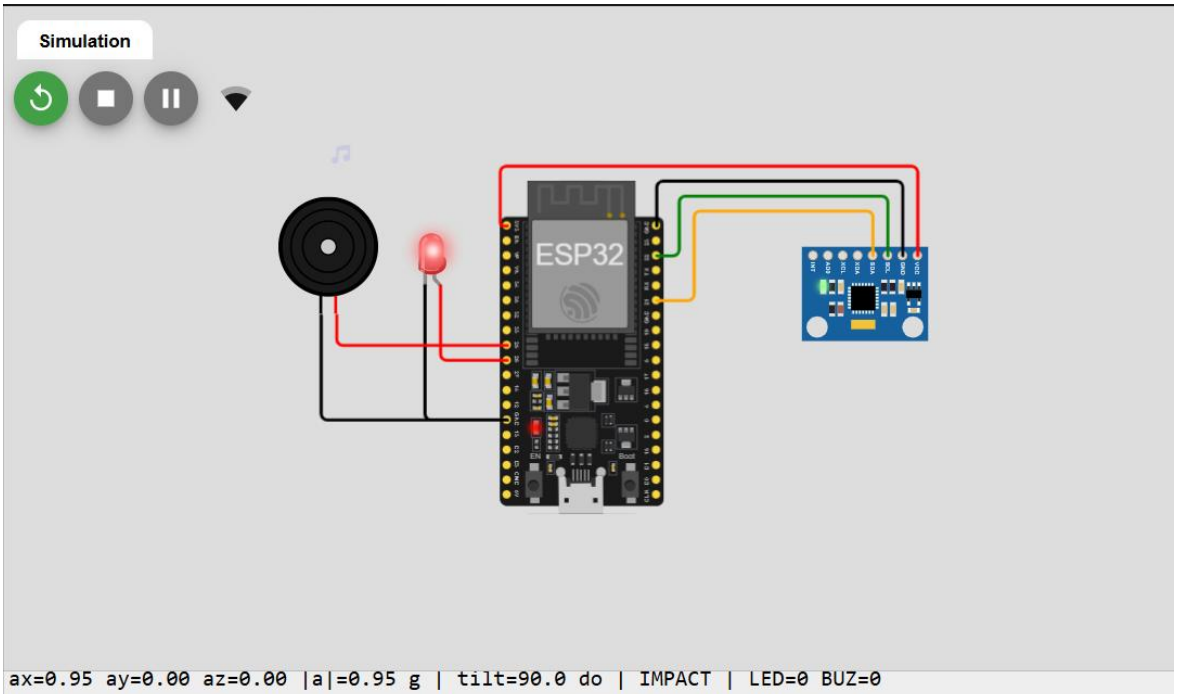

```
ax=0.10 ay=-1.50 az=1.45 |a|=2.09 g | tilt=46.0 do | IMPACT | LED=0 BUZ=0
ax=0.10 ay=-1.50 az=1.45 |a|=2.09 g | tilt=46.0 do | IMPACT | LED=0 BUZ=0
ax=0.10 ay=-1.50 az=1.45 |a|=2.09 g | tilt=46.0 do | IMPACT | LED=0 BUZ=0
ax=0.10 ay=-1.50 az=1.45 |a|=2.09 g | tilt=46.0 do | IMPACT | LED=0 BUZ=0
ax=0.10 ay=-1.50 az=1.45 |a|=2.09 g | tilt=46.0 do | IMPACT | LED=0 BUZ=0
ax=0.10 ay=-1.50 az=1.45 |a|=2.09 g | tilt=46.0 do | IMPACT | LED=0 BUZ=0
ax=0.10 ay=-1.50 az=1.45 |a|=2.09 g | tilt=46.0 do | IMPACT | LED=0 BUZ=0
```



c. Trạng thái ngã tự do



d. Trạng thái té ngã




```
ax=0.95 ay=0.00 az=0.00 |a|=0.95 g | tilt=90.0 do | IMPACT | LED=0 BUZ=0
ax=0.95 ay=0.00 az=0.00 |a|=0.95 g | tilt=90.0 do | IMPACT | LED=0 BUZ=0
!!! PHAT HIEN TE NGA !!!
ax=0.95 ay=0.00 az=0.00 |a|=0.95 g | tilt=90.0 do | FALL | LED=1 BUZ=1
ax=0.95 ay=0.00 az=0.00 |a|=0.95 g | tilt=90.0 do | FALL | LED=1 BUZ=1
ax=0.95 ay=0.00 az=0.00 |a|=0.95 g | tilt=90.0 do | FALL | LED=1 BUZ=1
ax=0.95 ay=0.00 az=0.00 |a|=0.95 g | tilt=90.0 do | FALL | LED=1 BUZ=1
```

